

**1. Install VirtualBox/VMware Workstation with different flavors of linux or windows OS on top of windows7 or 8.**

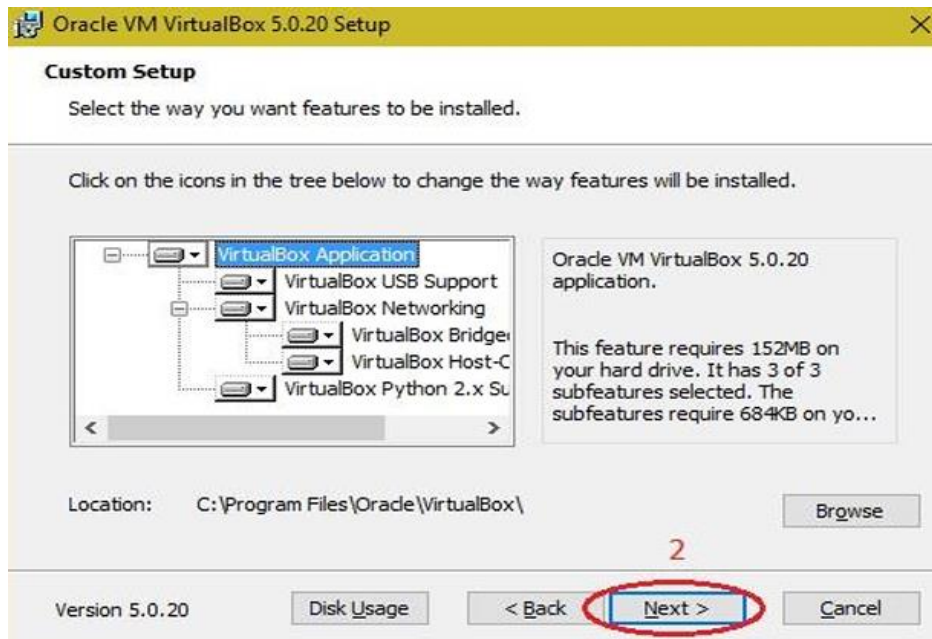
**Aim:** To Install Virtualbox / VMware Workstation with different flavours of linux or windows OS.

**PROCEDURE:**

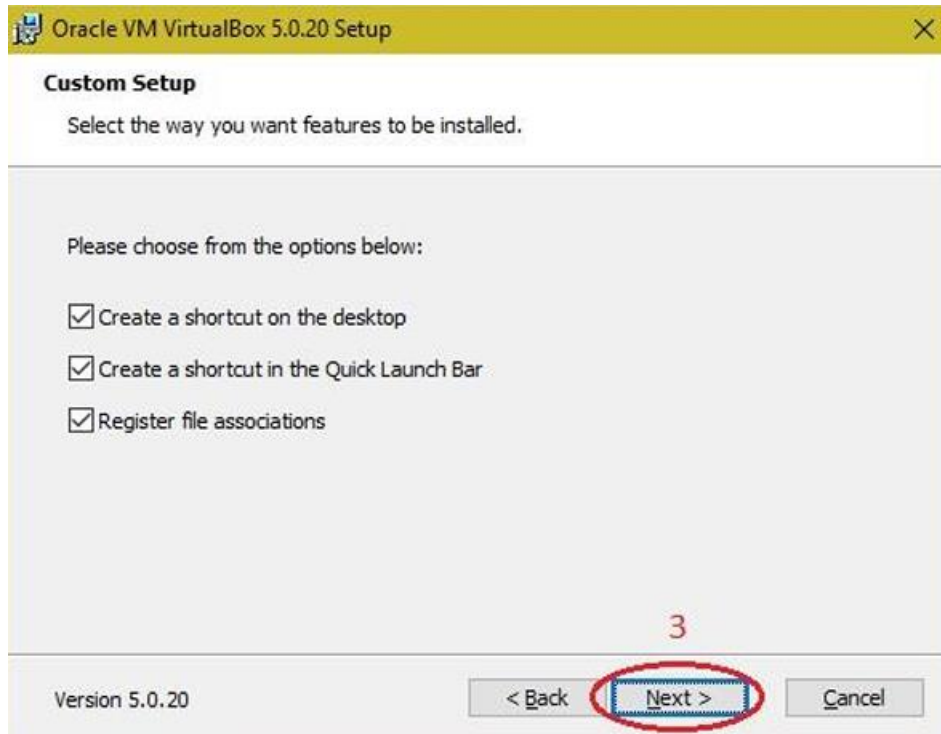
1. Download the Virtual box exe and click the exe file...and select next button.



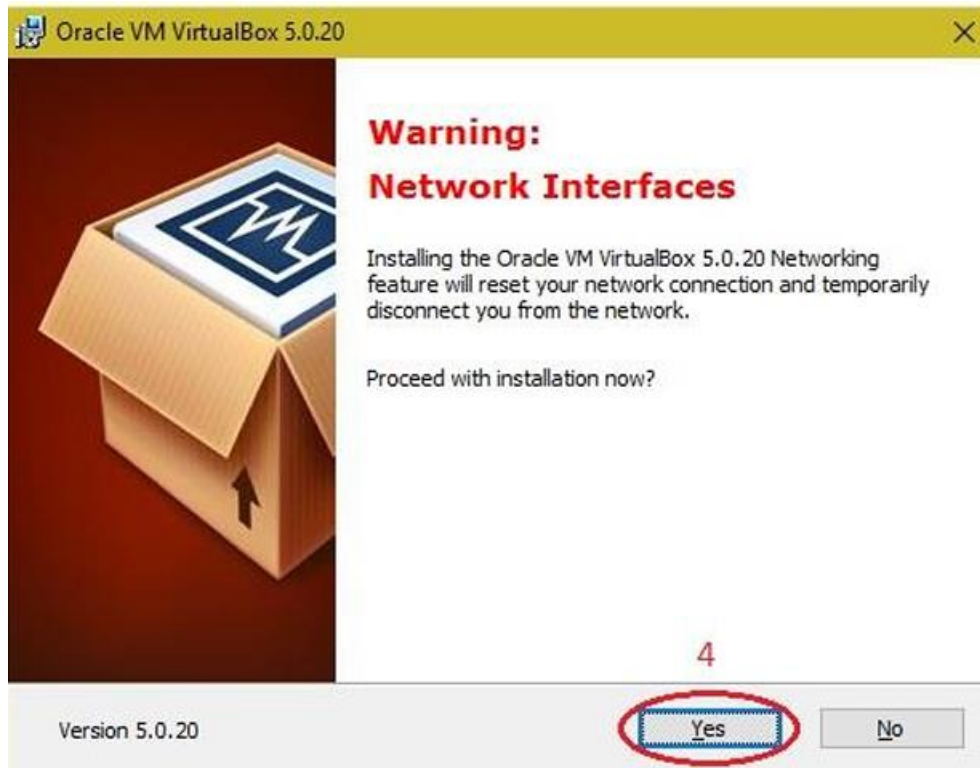
2. Click the next button.



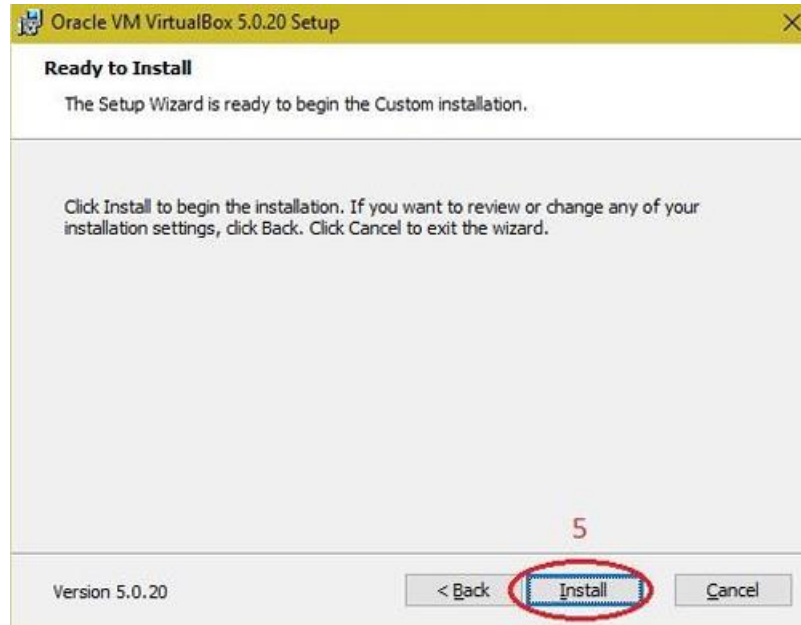
3. Click the next button.



4. Click the YES button.



5. Click the install button...



6. Then installation was completed. The show virtual box icon on desktop screen....



VirtualBox

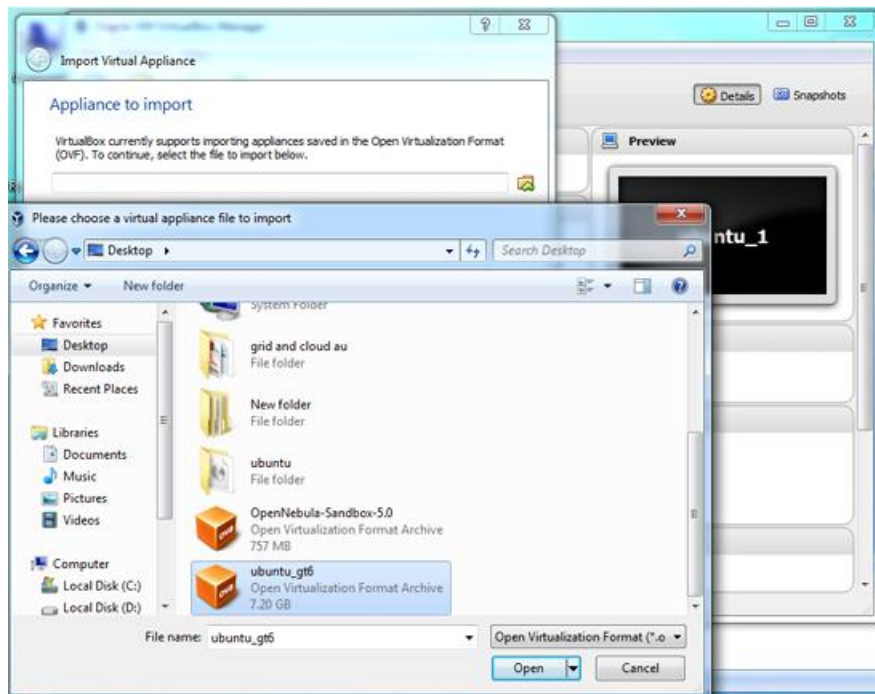
## 2. Install a C compiler in the virtual machine created using virtual box and executes Simple program.

**Aim:** To Install a C compiler in the virtual machine created using virtual box and executes Simple Program.

### PROCEDURE:

#### Steps to import Ubuntu file

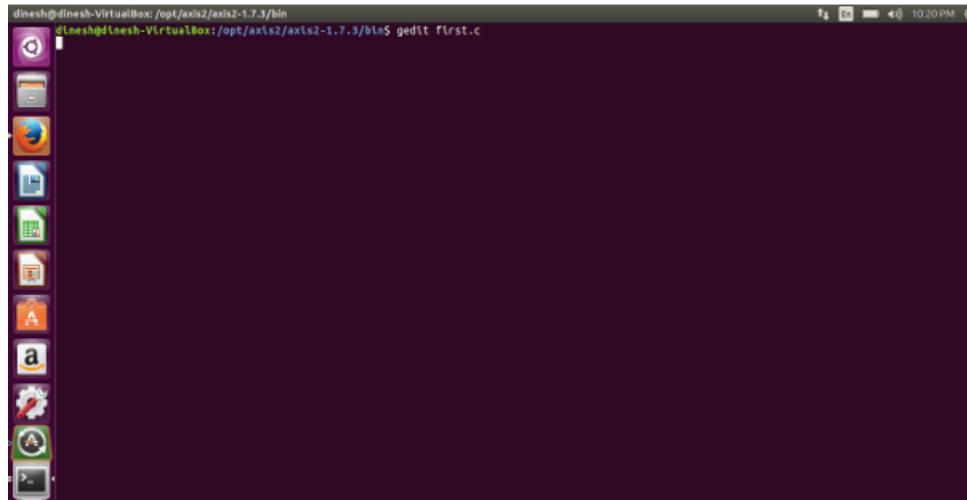
1. Open Virtual box
2. File  $\Rightarrow$  import Appliance
3. Browse ubuntu\_gt6.ova file
4. Then go to setting, select USB and choose USB 1.1
5. Then Start the ubuntu\_gt6



#### Steps to run C program:

1. Open the terminal
2. Type `cd /opt/axis2/axis2-1.7.3/bin` then press enter
3. `gedit hello.c`
4. `gcc hello.c`
5. `./a.out`

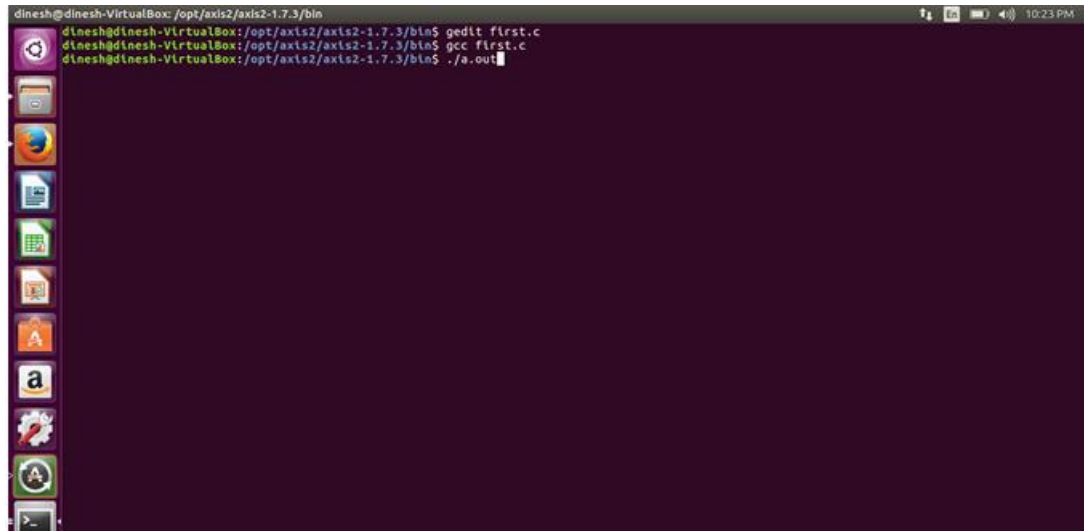
## Type gedit first.c



## Type the C program

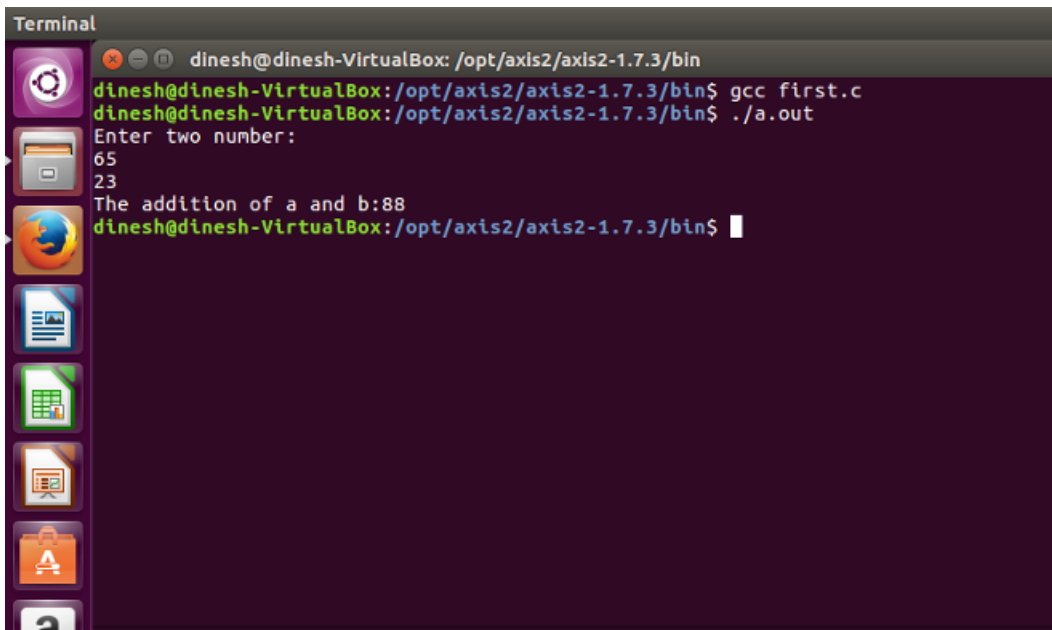


## Running the C program



```
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ gedit first.c
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ gcc first.c
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ ./a.out
```

## Display the output:



```
Terminal
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ gcc first.c
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ ./a.out
Enter two number:
65
23
The addition of a and b:88
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$
```

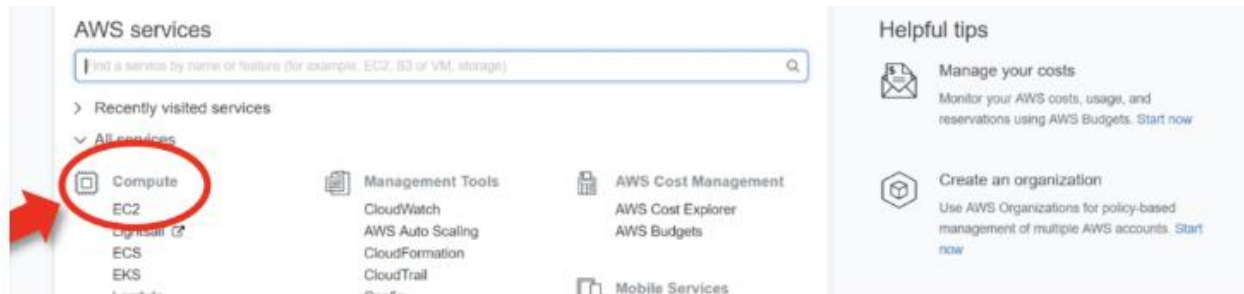
### 3. Create EC2 Instance in AWS (Amazon)

**Aim:** Create EC2 Instance in AWS (Amazon)

**PROCEDURE:**

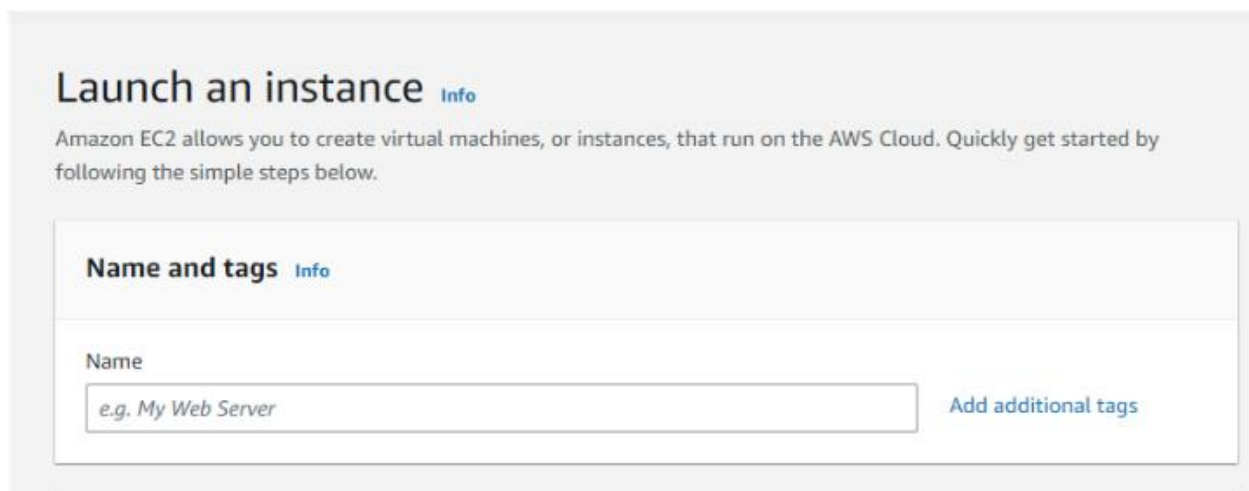
The following are the steps for creating an EC2 instance in AWS (Amazon):

**Step 1:** First, log into your [AWS account](#) and click on “services” present on the left of the AWS management console, i.e. the primary screen. From the drop-down menu of options, tap on “EC2”. To create an AWS free tier account refer to [Amazon Web Services \(AWS\) – Free Tier Account Set up.](#)

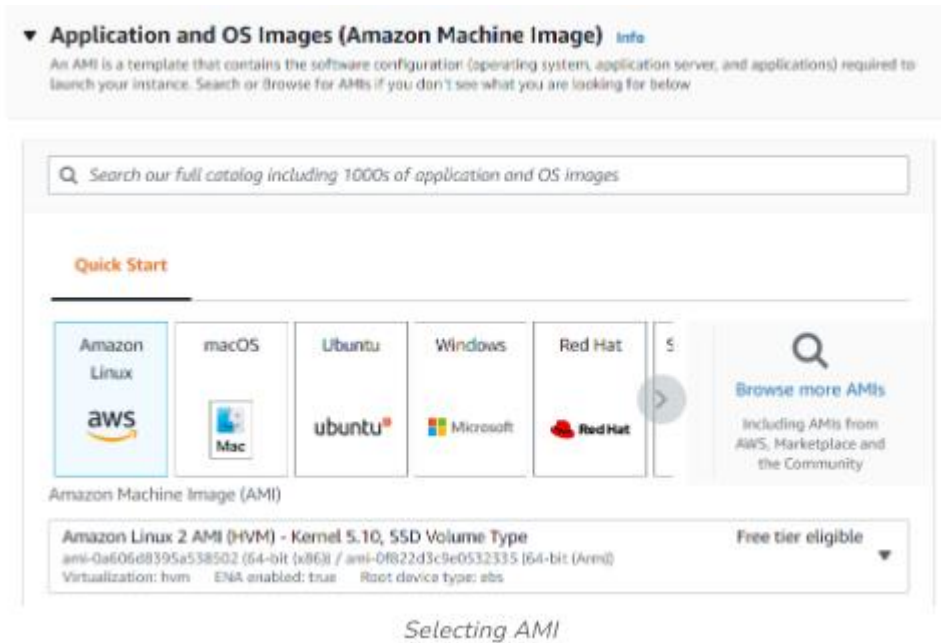


Under Resources >> Click on “Instances running” — It will show if any [EC2 instances](#) are running or not.

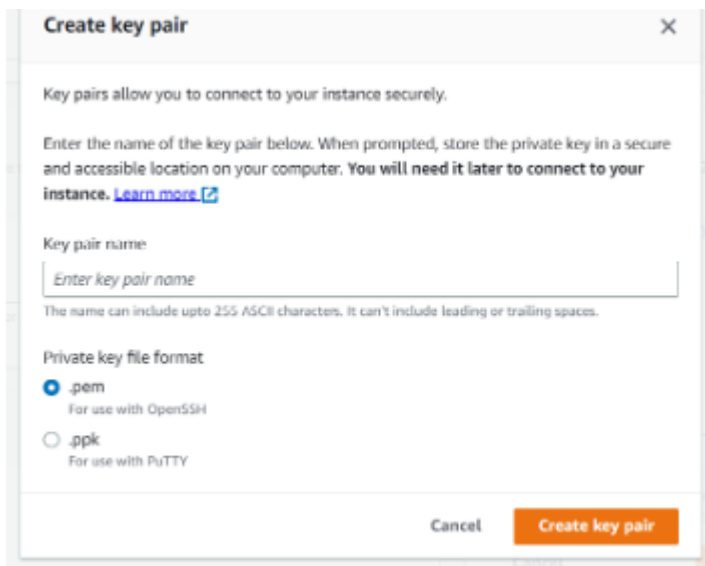
**Step 2:** Click on the launch instance click on the launch instance, after clicking on it you will be redirected to a launch page where we can create an instance. Configure all the requirements to Create a new instance like the name of the instance as shown in the figure below.



**Step 3:** Select AMI – Required operating system from the available. There are different types of OS available select the OS as per your requirement.



**Step 4:** By default, it selects a free tier of storage. (IF YOU ARE ELIGIBLE FOR THE FREE TIER). From the available storage specifications, select a free tier-eligible storage service. The instance type includes the number of CPUs required and the Memory required for your application. By default, the instance type is “t2.micro” which is a free tier-eligible service. Do not select any other which leads to the billing amount. To know more about instance types refer to [Amazon EC2 – Instance Types](#).



**Step 5:** Keep the network settings as default settings and make changes if required. Storage As mentioned in the picture, Free tier eligible can get up to 30 GB of [EBS Storage](#). Keep it as default.

▼ **Configure storage** [Info](#) Advanced

1x  GiB  ▼ Root volume (Not encrypted)

i Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage ✕

[Add new volume](#)

The selected AMI contains more instance store volumes than the instance allows. Only the first 0 instance store volumes from the AMI will be accessible from the instance

0 x File systems [Edit](#)

**Step 6:** Launching Instance At last, Check if all the selected are eligible for a free tier or not and click on “Launch instance”.That’s it, an instance will be created.

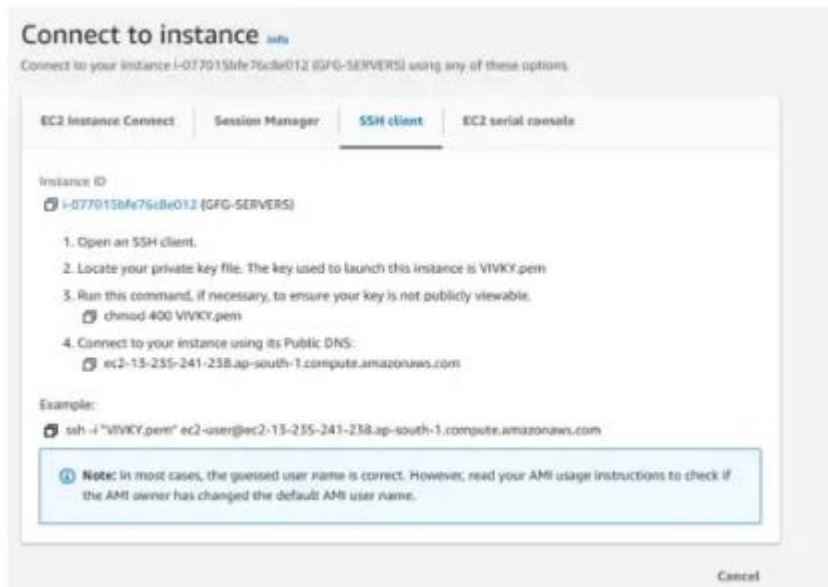
### Steps To Connect Terminal Using SSH-Key:

**Step 1:** Select the server to which you want to connect and click on the connect button at the top of that instance as shown in the image below.

Instances (1/1) [Info](#) [Refresh](#) [Connect](#)

| <input checked="" type="checkbox"/> | Name        | Instance ID         | Instance state                             | Instance type | Status check                                   | Alarm status | Availability Zone | Public IPv4 DNS         |
|-------------------------------------|-------------|---------------------|--------------------------------------------|---------------|------------------------------------------------|--------------|-------------------|-------------------------|
| <input checked="" type="checkbox"/> | GFG-SERVERS | i-077015bfe76c8e012 | <span style="color: green;">Running</span> | t2.micro      | <span style="color: gray;">Initializing</span> | No alarms    | ap-south-1a       | ec2-15-235-241-238.ap.. |

**Step 2:** Copy the SSH key which is right following the example it will acct as a [key-pair](#) to connect to EC2-Instance.



**Step 3:** Open the terminal and go to the folder where your .pem file is located and paste the key that you have copied in AWS and paste it in the terminal.

```
PS C:\Users\rknav\Downloads> ssh -i "VIVKY.pem" ec2-user@ec2-13-235-241-238.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-13-235-241-238.ap-south-1.compute.amazonaws.com (13.235.241.238)' can't be established.
ED25519 key fingerprint is SHA256:SVxqQUp4UBe9rUMXvZ1uL9UnzRNfpSFk8DjMybXVoyE.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? YES
Warning: Permanently added 'ec2-13-235-241-238.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-dashboard
[ec2-user@ip-172-31-34-45 ~]$
```

#### 4. Develop a simple applications using Apex programming language of salesforce.com

**Aim:** To develop a simple custom application using the Apex programming language on the Salesforce cloud platform.

##### PROCEDURE:

**Step 1: Sign up for a Salesforce Developer Org:** If you don't have one, sign up for a free developer account on the Salesforce Signup Page.

**Step 2: Log in to your Developer Org:** Access your Salesforce instance with your credentials.

**Step 3: Open the Developer Console:** Click the **gear icon** (Setup) in the top-right corner of the Salesforce Classic interface or the top-left in Lightning Experience, and then select **Developer Console**.

**Step 4: Create a New Apex Class:**

1. In the Developer Console, navigate to **File > New > Apex Class**.
2. Enter a name for your class (e.g., HelloWorldApp) and click **OK**.

**Step 5: Write the Apex Code:** A basic class structure will be automatically generated. Add a method within the curly braces to perform a simple action, such as printing a message to the debug log.

##### Apex Code

```
public class HelloWorldApp {  
    public static void sayHello() {  
        System.debug('WELCOME TO APEX PROGRAMMING');  
    }  
}
```

Ensure you click **File > Save** to save your new class.

**Step 6: Execute the Apex Code:**

1. In the Developer Console, click the **Debug** menu and select **Open Execute Anonymous Window**.
2. In the window that opens, enter the following code to call your method:

##### Apex Code

```
HelloWorldApp.sayHello();
```

3. Ensure the **Open Log** checkbox is selected and click the **Execute** button.

**Step 7: View the Output:** The execution log will open automatically. To see only your output, ensure the **Debug Only** checkbox is selected in the log inspector. Your message "WELCOME TO APEX PROGRAMMING" will be displayed in the log.

**5. Implement a mailing service using Apex programming language of salesforce.com**

**Aim:** Implement a mailing service using Apex programming language of salesforce.com

**Procedure:**

**Step 1:** Open the Developer Console To open the Developer Console, you can either click on “Your Name” or the quick access menu (Setup gear icon) on the top right corner of the Salesforce page.

**Step 2:** Create a new Apex class In the Developer Console, click on “File” and then “New” and select “Apex Class”. Enter “EmailManager” as the class name and then click “OK”.

**Step 3:** Replace the default class body Replace the default class body with the following example of the EmailManager class.

```
public class EmailManager {
    // Public method
    public void sendMail(String address, String subject, String body) {
        // Create an email message object
        Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
        String[] toAddresses = new String[] {address};
        mail.setToAddresses(toAddresses);
        mail.setSubject(subject);
        mail.setPlainTextBody(body);
        // Pass this email message to the built-in sendEmail method
        // of the Messaging class
        Messaging.SendEmailResult[] results = Messaging.sendEmail(
            new Messaging.SingleEmailMessage[] { mail });
        // Call a helper method to inspect the returned results
        inspectResults(results);
    }
}
```

```
private static Boolean inspectResults(Messaging.SendEmailResult[] results) {
    Boolean sendResult = true;
    // sendEmail returns an array of result objects.
    // Iterate through the list to inspect results.
    // In this class, the methods send only one email,
    // so we should have only one result.
    for (Messaging.SendEmailResult res : results) {
        if (res.isSuccess()) {
            System.debug('Email sent successfully');
        }
        else {
            sendResult = false;
            System.debug('The following errors occurred: ' + res.getErrors());
        }
    }
    return sendResult;
}
```

**Step 4:** Save the class Click on “File” and then “Save” to save your class. Once saved, you can use this class to send emails in other Apex classes or triggers.

### Step 6: Test the output

1. In the Developer Console, click **Debug | Open Execute Anonymous Window**.
2. In the window that opens, enter the following. Replace ‘Your email addresses with your email address.

```
EmailManager em = new EmailManager();  
em.sendMail('Your email address', 'Email Subject', 'Email Body');
```

Click Execute. Check your email!

If you want to call the method directly without using “new”, use the “static” keyword and call it as follows;

```
public static void sendMail(String ...  
...
```

```
EmailManager.sendMail('Your email address', 'Email Subject', 'Email Body');
```

## 6. Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.

**Aim:** To Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in Cloud Sim.

### PROCEDURE:

To use CloudSim in Eclipse:

**Step 1:** Download CloudSim installable files from

<https://code.google.com/p/cloudsim/downloads/list> and unzip

**Step 2:** Open Eclipse

**Step 3:** Create a new Java Project: File ->New

**Step 4:** Import an unpacked CloudSim project into the new Java Project

**Step 6:** Data centres are the resource providers in CloudSim; hence, creation of data centres is a second step. To create Datacenter, you need the Data center Characteristics object that stores the properties of a data centre such as architecture, OS, list of machines, allocation policy that covers the time or space shared, the time zone and its price:

**Step 7:** The third step is to create abroker:

**Step 8:** The fourth step is to create one virtual machine unique ID of the VM, userId ID of the VM's owner, mips, number of PEs amount of CPUs, amount of RAM, amount of bandwidth, amount of storage, virtual machine monitor, and cloud let Scheduler policy for cloudlets:

**Step 9:** Submit the VM list to the broker: `broker.submitVmList(vmlist)`

```
Cloudlet cloudlet = new Cloudlet(id, length, pesNumber, fileSize, outputSize, utilizationModel,
utilizationMode
```

**Step 12:** Start the simulation: `CloudSim.startSimulation()`

Starting

Initialising...

[illegible]

starting...

Entities started.

: Broker: Cloud Resource List received with  
1resource(s) 0.0: Broker: Trying to Create VM #0  
inDatacenter\_0  
: Broker: VM #0 has been created in Datacenter #2, Host#0  
0.1: Broker: Sending cloudlet 0 to VM #0  
400.1: Broker: Cloudlet 0 received  
: Broker: All Cloudlets executed. Finishing....400.1: Broker:  
Destroying VM#0  
Broker is shutting down...  
Simulation: No more future events  
CloudInformationService: Notify all CloudSim entities for shutting down. Datacenter\_0 is shutting  
down...  
Broker is shutting down...  
Simulation completed.  
Simulation completed.  
=====OUTPUT=====

| Cloudlet ID | STATUS  | Data center ID | VM ID | Time Start | Time Finish |
|-------------|---------|----------------|-------|------------|-------------|
| 0           | SUCCESS | 20             | 400   | 0.1        | 400.1       |

\*\*\*\*\*Datacenter: Datacenter\_0\*\*\*\*\* Userid Debt  
3 35.6  
CloudSimExample1 finished!